

超体宇宙城游戏软件

软件设计说明书

版本号：V1.0

1 软件概述

竖屏2D像素风城市模拟小游戏。画面呈现一座城市，中心是超体脑塔，三个城区环绕，居民在街区活动，机器人在道路上行走代表机队AI劳动，天气和昼夜实时变化，各类事件通过气泡和灯光脉冲呈现。城市数据来自外部状态文件，由感知器只读轮询驱动，不连服务器、不写外部。

团结引擎C#开发，逻辑在`FluxVerse`命名空间下。模拟规则写成纯C#静态规则类，数据与几何分离，居民个人数据放JSON、C#只管座位和布局规则。

2 运行环境

项	要求
操作系统	Android 8.0及以上 / iOS 12.0及以上
运行平台	微信小游戏、抖音小游戏、移动端
游戏引擎	Unity 2021.3 LTS (团结引擎1.10.3)
渲染管线	Built-in 2D, SpriteRenderer
编程语言	C#
运行内存	2GB以上
存储空间	150MB以上
屏幕分辨率	720x1280以上，竖屏

3 工程结构

代码在`City/Assets/Scripts/`，按职责分族：

```
Scripts/  
├─
```

		AmbientWeather.cs	24h
		CityAmbient.cs	MonoBehaviour
		CityLabsTemporal.cs	
		EventRouter.cs	
		CityEventRouter.cs	MonoBehaviour
		ResidentRules.cs	32
		ResidentIdentity.cs	JSON
		ResidentTags.cs / ResidentLabelRules.cs / ResidentLabelsUI.cs	
		ResidentBarks.cs / ResidentBubbles.cs / CityBubbles.cs	
		CityResidentCard.cs / ResidentCardRules.cs	
		RobotRules.cs	
		VehicleRules.cs	
		CityStreetBehavior.cs / StreetBehaviorRules.cs	
		FacadeRules.cs / OfficeRules.cs / BrainCrownRules.cs	
		CityInterior.cs / InteriorWindow.cs	
		LightFxRules.cs / CityLightFx.cs	
		WindowLightRules.cs / CityWindowLight.cs	
		RimLightRules.cs / NeonSigns.cs	
		WaterFxRules.cs / CityWaterFx.cs	
		UI	
		CameraRig.cs / CityCameraRig.cs	
		UIKit.cs / TDDemo.cs	
		MoodDirector.cs / MoodVisualRules.cs	
		AudioRouter.cs / CityAmbientAudio.cs	
		LabsRules.cs / LabsTemporalRules.cs	

组件类与.cs文件名一一对应，否则保存的场景引用会跨编辑器会话失效，所以 MonoBehaviour 适配器单独成文件，纯逻辑核心留在无头可测的文件里。

4 感知器与数据接入

AmbientWeather 和 EventRouter 负责外部数据接入。城市运行状态读`world/world-state.json`，事件序列读事件流文件，全部只读轮询、间隔约10秒，引擎从不写world/ 或任何同级仓库，所有写操作归感知器扫描所有。

world-state.json 消费的扁平键：city_day_phase（城市昼夜相位）、beijing_hhmm（北京时间）、weather_kind、weather_code、weather_wind_ms（风速）。读入后解析、与上一状态比对，把变化分发给事件路由和各模块。文件缺失时退回演示状态（按本机时钟和默认天气），城市照常运行。

5 城市布局与建筑

城市以中心超体脑塔为核心，周围划分QUANT、GAME、MEDIA三个功能区，另有北侧步道和南侧街道广场。

建筑规则类管立面（FacadeRules）、办公室（OfficeRules）、脑塔冠顶（BrainCrownRules）的生成和状态。建筑按网格程序化摆放、保证道路连通，建筑状态变化时切换灯光和配色。

6 居民座位规则

ResidentRules 定义32个居民座位，32x32像素、PPU24，半边长0.6667单位。居民按区分摊：

分区	QUANT	GAME	MEDIA	北侧	脑塔	访客
数量	9	7	8	2	1	5

座位覆盖：南侧QUANT/GAME/MEDIA广场铺装行、南侧街道南人行道、北侧步道（塔脚广场）、脑塔两侧一个荣誉座、以及南街上的5个跨城访客（座位不等于身份归属）。座位表提交前经过沙盒座位测试、数千条断言。

分区判定 ZoneOf 按序号区间返回，身份证明据此重新推导城区面：

```
static string ZoneOf(int i)
{
    if (i < QuantCount) return "QUANT";
    if (i < QuantCount + GameCount) return "GAME";
    if (i < QuantCount + GameCount + MediaCount) return "MEDIA";
    ...
}
```

7 纸娃娃部位堆叠

每个居民是纸娃娃部位堆叠，父物体不挂渲染器，每个身体部位是一个子SpriteRenderer，从九宫格图集切出：

部位堆叠顺序	pant → skin → cloth → badge → hair → eyes
部位z步进	0.05，裤子最深、眼睛最前
街道排序层	order 7（招牌6 < 街道7 < 染色8）

同排序层下Unity按相机距离排，裤子最深、眼睛最近。各层按身份调色。图集空格（生产线尚未画的衣服、工牌）无害挂载，堆叠对后续填图向前兼容。每个居民带接地阴影，32x8、阴影中心在脚线下方0.06。

8 居民身份数据分离

数据分离是硬规则：C#只拥有座位几何（物体名+世界坐标），每个人的身份、部件、图板索引、调色全部在`Assets/Data/residents-street.json`，由ResidentIdentity直接解析，C#不拷贝任何个人数据，从结构上杜绝副本漂移。数据文件缺失或损坏时Load返回空，运行时读空即“此版本身份缺席”、保持沉默。

一个街道槽的完整身份记录 ResidentIdentityEntry 字段：

字段	含义
slot / go	槽索引0-31、场景物体名
zone / district	座位分区、归属城区（荣誉座为空）
id / name	人口id、名字
species	carbon / silicon / sprite
gender / age	性别、年龄（-1为未公开）
faction / block	阵营、所属街区
profession / axis	职业、思想轴
creed / layer	信条、层级（narrative，荣誉座为anchor）
hairPart / eyePart	发型部件、眼型部件
pantC/skinC/hairC/clothC/badgeC/eyeC/coreC	各部位十六进调色
plateIndex / plateName	名牌图板行序、名牌文字

归属律：城市座位标QUANT/GAME/MEDIA/北侧，脑塔荣誉座城区为空，访客座带着真实归属城区、人却站在南街上一——座位不等于身份归属。诚实律：人口派生槽层级都是narrative，人类来源的荣誉座标anchor。

居民另有独立的口头禅（ResidentBarks）、气泡（ResidentBubbles/CityBubbles）、身份标签（ResidentTags/ResidentLabelRules/ResidentLabelsUI）和名片卡（CityResidentCard/ResidentCardRules），点居民可弹名片看身份信息。

9 街道机器人

RobotRules 定义8个街道机器人，16x16像素、PPU16，零重采样。机器人在道路网格上行走，带行走帧动画，帧表16槽含站立、并腿、迈步等姿势，每台分配一个不同的可用帧，不出现克隆行。

机器人代表机队AI劳动：行走、停留、工作状态对应真实劳动行为，位置和状态由外部状态驱动。机器人保留道路行走域（道路对它们是合法站立地，只有居民约束在人行道），每台带接地阴影。

10 载具与街道行为

VehicleRules 管载具通行，StreetBehaviorRules 和 CityStreetBehavior 管街道行为规则，站立地从实时 tilemap 重新推导“站在地块/道路格上”，而不是信任注释。

11 昼夜色轮

AmbientWheel 维护24小时四档环境色轮，时间映射真实时钟，档位边界对齐时钟探针：

档位	小时区间
Dawn 黎明	5-8
Day 白天	8-17
Dusk 黄昏	17-20
Night 夜晚	其余

每档调色板含天空顶色、天空底色、环境染色、染色透明度、相机背景：

档位	天空顶	天空底	特征
黎明	冷蓝	暖粉地平线	清晨过渡
白天	天蓝	淡蓝	地块全色
黄昏	紫	暖金地	紫金暮色
夜晚	深蓝黑	深蓝	青色来自城市灯

粉色紫色只作环境天空染色，永不作功能性灯光；夜晚城市灯带电青色。

调色板在代码里逐档写死，每档给出天空顶底、染色和透明度：

```
AmbientPalette PaletteFor(AmbientTier t)
{
    AmbientPalette p = new AmbientPalette();
    switch (t)
    {
        case Dawn: // cool blue zenith over warm pink horizon
            p.skyTop = new Color(0.42f, 0.52f, 0.72f);
            p.skyBottom = new Color(0.98f, 0.68f, 0.52f);
            p.tint = new Color(1f, 0.72f, 0.55f);
            p.tintAlpha = 0.08f;
            break;
        case Day: // clean daylight, tiles at full color
            p.skyTop = new Color(0.45f, 0.66f, 0.92f);
            p.skyBottom = new Color(0.78f, 0.87f, 0.96f);
            p.tint = Color.white; p.tintAlpha = 0f;
            break;
        case Dusk: // purple zenith, warm gold horizon
            p.skyTop = new Color(0.36f, 0.22f, 0.48f);
            p.skyBottom = new Color(0.98f, 0.58f, 0.32f);
            p.tint = new Color(1f, 0.62f, 0.42f);
            p.tintAlpha = 0.22f;
            break;
        default: // night: deep blue-black, cyan from city lights
            p.skyTop = new Color(0.015f, 0.02f, 0.06f);
            p.skyBottom = new Color(0.05f, 0.09f, 0.20f);
            p.tint = new Color(0.04f, 0.09f, 0.24f);
            p.tintAlpha = 0.45f;
```

```
break;
}
p.camBg = p.skyTop;
return p;
}
```

状态文件缺失时按主机当前小时兜底定档，保证开机一定有一个有效色轮。

12 档位切换混合

色轮的硬切是观感缺陷，所以档位翻转走调色板混合。AmbientBlend 在2.5秒内缓动五值族（天空顶底、染色rgb、透明度、相机背景）：

```
float k = Clamp01(elapsed / seconds);
float s = RigMath.EaseInOut(k); //
p.skyTop = Color.Lerp(from.skyTop, to.skyTop, s);
```

应用环境保留即时 settle 原语供启动构建，游玩路径走平滑过渡，完成时吸附到目标。

13 天气规则

天气模式分

None、Rain、Snow，每类有独立粒子、色轮和特效规则，切换走过渡，天气数据可接真实天气。

警报律对齐天气探针：风速达到17.2m/s（大风族）或重度WMO天气码，触发全城红色警报带（功能性红色）。天气还驱动雾效和环境音。

14 事件路由

FluxEventRouter 是事件纯逻辑核心，轮询/游标/解析/路由。FluxEvent 结构字段：

字段	含义
ts_utc	时间戳
type	事件类型
actor	执行者
repo / zone	来源仓库、城区
summary	摘要

轮询间隔10秒，cursor 记已消费的流行数。启动时 SeekToEnd 跳到末尾，只让引擎启动后到达的事件触发脉冲，历史永不回放。活动流每日归档自愈，若行数小于光标则光标归零。

Tick 里先到点轮询、再推进所有脉冲和特效的生命周期，过期即移除：

```
void Tick(float dt)
{
pollTimer += dt;
if (pollTimer >= PollIntervalSec)
```

```

{
pollTimer = 0f;
PollOnce();
}
for (int i = pulses.Count - 1; i >= 0; i--)
if (!pulses[i].Advance(dt)) pulses.RemoveAt(i);
for (int i = fx.Count - 1; i >= 0; i--)
if (!fx[i].Advance(dt)) fx.RemoveAt(i);
if (breath != null && !breath.Advance(dt)) breath = null;
}

```

PollOnce 从游标处读新增行、逐行解析成 FluxEvent、按类型路由到对应锚点的光脉冲和特效。能进路由闸的类型集中在 MappedTypes 一处维护，视觉面和音频映射共用同一集合、不会各写各的漂移。未登记类型不触发任何效果。

事件分发后还可挂一个 EventSink 回调，未接线时空操作，保持只跑脉冲的基线字节不变。

15 事件脉冲与特效

事件分发后在对应锚点触发 GlowPulse 灯光脉冲和 TransientFx 特效。CEO指令类事件触发脑塔白色天线脉冲（白色，五色律），并呈现五个城市核心面：操作系统轮次呼吸、城市验证门、机队运输带。

EventSink 在每次分发事件、脉冲之后触发，未接线时空操作、保持基线一致。锚点查找通过回调把事件映射到世界坐标。

16 呼吸光与情绪

BreathGlow 每个操作系统轮次一个，轮次开始淡入、结束淡出。呼吸环峰值受 MoodDirector 调制，MoodVisualRules 闭合带0.70到1.15，默认1.0保持基线零漂移；设置时钳制到带宽并立即重挂活的呼吸。

情绪导演 MoodDirector 从和城市气泡相同的世界输入里推导出整体情绪，是独立的同路径读取、互不耦合。输入含分钟级时钟、24小时事件密度、天气类型、天气警报和日历节日行。情绪是一个封闭五态集合，优先级为肃穆高于节庆、节庆高于活泼、活泼高于安静、安静高于平稳，负信号先判，全程走确定性摘要、不接语言模型。

情绪只在时钟层移动氛围桶的选择，不会改写事件和天气这些事实门上下文，也不改写居民个人人设。深夜窗口里整体情绪偏安静，节日行按月日每年命中、按年月日仅当年命中，没有来源指针的日历行一律拒绝、不凭空发明节日。

情绪经 MoodVisualRules 映射到两个闭合带通道：一是被调制的霓虹招牌做运行时灰度染色（豁免的天线结构不动），二是路由呼吸环峰值。场景保存前先把情绪恢复到中性白、系数1.0，避免把临时情绪持久化。

16.1 城市环境视觉面

城市环境总控

CityAmbient

除了跑昼夜色轮，还管一批运行时视觉面，它们按需创建、不存进场景，释放时统一回收。

黄昏时屋顶轮廓亮起一排顶边光带，alpha 随黄昏衰减强度、其余时段硬归零。黎明和黄昏各有一块地平线发光四边形，颜色取插值调色板的天空底色、透明度随同一缓动曲线，不做硬切。黄昏时北侧岸边还会罩一层淡紫的深度雾纱，做空气透视。

环境染色带分成城南、河面、城北三块。河面块在黎明和黄昏豁免暖色染色——蓝紫水面在暖染色和线性颜色空间里会读成玫瑰色、不自然，所以河面走自己的透明度窗口，随夜晚同样的缓动暗下去，城南北块保持四档色轮律不变。

17 内景

CityInterior 是内景窗口的薄适配器，纯核心 InteriorRouter 留在 InteriorWindow.cs。运行时左键点一个已注册建筑，即打开该公司的实时面板（Windows目标没有原生WebView，走系统打开），同时引擎内弹出一块玻璃横幅作确认。

横幅是程序化 SpriteRenderer 堆叠，不用 uGUI Canvas——世界空间Canvas在批处理渲染里会退化成屏幕空间覆盖层、位置错乱，程序化精灵在批处理验证和运行时表现一致。横幅外壳走 Uikit 玻璃面板配方：暗玻璃渐变、细冷色边、蓝紫外发光。

横幅参数	值
生命周期	6秒
世界尺寸	20 × 2.6
排序层	光20 / 边21 / 玻璃22 / 文字23

横幅是运行时按需创建、永不存进场景，隐藏时释放它生成的所有运行时纹理。中文文案预烘焙成图片再作为运行时精灵层加载（引擎内无中文字体资源）。InteriorWindow 另管窗内动态，内景数据同样来自外部状态。

18 灯光与水面

灯光族（LightFx、WindowLight、RimLight、NeonSigns）统一管夜间亮灯、自发光招牌和轮廓光，昼夜和事件驱动开关。WaterFx 管水面动画和反光。特效规则集中在规则类、表现层按规则渲染。

霓虹招牌 NeonRules 定义21个静态招牌，挂在层order 6（道具4 < 招牌6 < 环境染色8），是建成城市的静态布景、和 tilemap 一样持久化在场景里。四种挂载：立面招牌嵌在建筑立面内、屋顶牌沉入天际线、街道家具站在铺装带、豁免类为脑塔天线。

六块公司牌按位置分布：FLUX和CPH4在脑塔玻璃立面，BIGGAME在GAME西屋顶，BIGMONEY在QUANT塔顶，BIGSTREAM在MEDIA东北矮楼，BIGLIFE在西北矮楼。

五色放置律：

颜色	对应
白	核心
蓝	超体
青	数据
金	资本
品红	流动

琥珀和招牌原包的粉青只作环境景色、永不作功能性灯光，粉色只用于环境天空。招牌 localScale 恒为1、零重采样，世界尺寸走每牌导入PPU档（大招牌用PPU32/48呈现半高/三高），招牌高度不超挂载建筑一半、不越过立面顶。

功能性灯光（警报带、事件脉冲）排在环境层之上，在静态城市画面上打出。窗户灯光 WindowLight 按入住和昼夜逐窗亮灭，轮廓光 RimLight 勾建筑边缘。

19 相机

CameraRig 是双模式相机，正交、PPU16，纯核心无头可测，CityCameraRig 是 MonoBehaviour 适配器。两档视野：

档位	正交size	视野	用途
L0 全景	20	71x40 tile	整条河湾城市、慢漂移
L1 街道	9	32x18 tile	建筑立面、机器人、窗内

档位切换在1.2秒内对 size 和位置同时缓动插值，禁止硬切，缓动走两端零速度 smoothstep。

L0全景带缓慢漂移，用两个不同周期的正弦（x周期48秒、y周期37秒、不同步）产生有机的轻微移动。ClampFocus 把相机中心钳制在绘制城市带内（y -16到14、x半宽34），保证街道档视野不越出已绘制区域。触控可缩放平移，预设视角一键切换。

20 音频

音频路由 FluxAudioRouter 是事件类型到音频片段的纯核心，不持有播放器、无头可测。它只给真实事件流实际发出、且片段已在包内的事件类型配音：

事件类型	片段
CEO_ORDER	指令脉冲
COMMIT	Laser激光
TASK_CLAIM	Robot_Activated机器人启动
MARKET_OPEN / CLOSE	开市/收市铃
WEATHER_ALERT	红色警报
RESIDENT_SAY	Robot_Talk机器人说话

还没有真实事件源的类型一律不配音，声音只锚定真实事件、不做装饰，未映射类型保持静音、不兜底放jingle。音量分两层，签名层1.0、事件音效层0.8。

环境床 FluxAmbientBed 分三路：按城市时段的四档背景音乐、按天气类型的天气层、以及常驻的城市噪声底床，BGM和环境音分通道调音量。

20.1 实验室时序面

CityLabsTemporal 是实验室时序面的适配器，所有决策由纯规则核心 LabsTemporalRules 掌握，适配器只做接线。它每10秒只读轮询世界状态，并维护事件流文件的尾游标，首次轮询定位到文件末尾、不回放历史，文件轮转时游标自查归零。

时序面分三个设施：诞生站给半活读数、六槽滚动墙和诞生仪式，沙盒给活读数和两个提案脉冲，孵化器保持待机暗。衬底四边形、数字行、墙覆盖和脉冲覆盖都是运行时子对象、带运行时精灵，数字行在运行时按字形位置逐个切图生成，因为活值是状态数据、不能做成烘焙资源。

普查段缺失时诞生读数隐藏、墙覆盖全关，演进段缺失时沙盒读数隐藏，设施精灵保持原样，是诚实缺席、不放假数字。诞生仪式不按定时器跑，它等待事件流上一个真实的居民诞生事件才触发。场景保存前销毁所有运行时子对象、保持磁盘纯净。

21 存档

保存视图偏好、相机预设、设置项，JSON本地存储。外部状态不存档、由感知器实时读取。自动保存加版本校验。

22 容错与性能

- 外部文件缺失或解析失败用演示数据
- 图集空缺部位无害挂载，不报错
- 资源缺失用占位，全局异常捕获
- 静态合批、对象池、按需加载
- 站立地从实时地块推导，布局规则经沙盒断言验证
- 包体20MB内

城市的时间、天气、事件全部来自外部状态文件的只读轮询，引擎本身不写任何外部数据，同一批输入在任何机器上呈现同样的城市。渲染层吃这些状态驱动昼夜、天气、居民和特效，数据和表现分得很清楚。

居民座位是几何事实、身份数据另存，纸娃娃按身份填图，从结构上杜绝改了身份却对不上座位的漂移。机器人、车辆、招牌各有独立规则和排序层，新增表现只在对应层动、不影响已有画面。

事件路由用游标只读新增、不回放历史，脉冲和音频按真实事件触发、不做无来源的装饰。城市是一个被动呈现外部真实状态的活体，而不是封闭的预制场景。

23 版本信息

项	内容
软件全称	超体宇宙城游戏软件
软件简称	超体宇宙城
版本号	V1.0
开发完成日期	2026年09月30日
发表状态	未发表
开发方式	独立开发
权利取得方式	原始取得
权利范围	全部权利
编程语言	C#
源程序量	约8300行